

План обучения Go Backend

2 часа • Учеба только в выходные • После каждого занятия выполнить домашнее задание

Дата	Тема	Таймкод	Дополнительно
05.07	Concurrency (большой пример)	05:22:18 — 06:10:45	Домашнее задание. Повторить: • Context • Mutex • RWMutex • WaitGroup  Дополнительно (10–15 минут): "Go WaitGroup Explained"  Мини-тест 1. Что делает WaitGroup? 2. Для чего нужен Mutex? 3. Когда возникает Race Condition? 4. Что делает Context? 5. Чем RWMutex отличается от Mutex?  Собеседование • Что такое goroutine? • Как работает WaitGroup? • Что такое deadlock?
06.07	HTTP Postman	06:10:45 — 07:03:49	Домашнее задание. Создать первый HTTP сервер самостоятельно.  Дополнительно 15 минут HTTP за 15 минут  Мини-тест 1. Что такое HTTP? 2. Что такое клиент? 3. Что такое сервер? 4. Что такое Request? 5. Что такое Response?



Собеседование

- Какие HTTP методы знаешь?
- Чем GET отличается от POST?
- Что такое статус-код?

07.07 Первый HTTP сервер
Request Body

07:03:49 — 08:13:12

Домашнее задание.

Самостоятельно написать сервер без видео.

Попробовать отправить запрос через Postman.



Мини-тест

1. Что такое Handler?
2. Как получить Request Body?
3. Что такое ResponseWriter?
4. Как вернуть JSON?
5. Как вернуть ошибку?



Собеседование

- Как работает net/http?
- Что делает ListenAndServe?
- Как написать простой API?

ИТОГ НЕДЕЛИ

К концу недели необходимо уверенно понимать:

- Goroutines
- Channels
- Mutex
- RWMutex
- WaitGroup
- Context
- HTTP
- Request
- Response
- Postman

Если какая-либо тема вызывает затруднения — сначала повторить её, затем переходить дальше.

10.07 Response
Status Codes
Headers

08:13:12 — 08:31:02

Домашнее задание.

Самостоятельно вернуть:

- 200

- 400
- 404
- 500

 Мини-тест

1. Что такое Header?
2. Что такое Content-Type?
3. Для чего нужен Status Code?

14.07	HTTP Methods JSON	08:31:02 — 09:25:16	Домашнее задание. Написать небольшой API, который принимает JSON и возвращает JSON.  Собеседование • Что такое JSON? • marshal/unmarshal? • Когда использовать PUT? • PATCH?
-------	----------------------	---------------------	--

15.07	Query Parameters REST API (часть 1)	09:25:16 — 10:35:00	Домашнее задание. Добавить Query-параметры в свой HTTP-сервер. Например: • /users?id=1 • /products?page=2  Дополнительно (15 минут) REST API за 15 минут  Мини-тест 1. Что такое Query Parameters? 2. Чем они отличаются от Path Parameters? 3. Что такое REST? 4. Что означает CRUD? 5. Когда использовать GET?  Собеседование • Что такое REST API? • Какие HTTP методы относятся к CRUD? • Что такое идиоматичность?
-------	--	---------------------	--

Написать небольшое API:

- GET /users
- GET /users/{id}
- POST /users
- DELETE /users/{id}

? Мини-тест

1. Что такое Endpoint?
2. Что такое Resource?
3. Что такое URI?
4. Почему REST называют архитектурным стилем?
5. Что возвращает POST?



Собеседование

- Назови принципы REST.
- Когда использовать POST, PUT и PATCH?
- Что такое Stateless?

ИТОГ НЕДЕЛИ

До перехода к следующему видео необходимо знать:

- HTTP полностью
- Postman
- JSON
- Headers
- Status Codes
- HTTP Methods
- REST API
- Query Parameters

✓ Видео №2 полностью закончено.

Создать GitHub-аккаунт (если его нет).

Создать первый локальный репозиторий.

? Мини-тест

1. Что такое Git?

2. Что такое Repository?
3. Что такое Commit?
4. Что такое Branch?
5. Что такое Merge?



Собеседование

- Зачем нужен Git?
- Разница между Git и GitHub?
- Что такое commit?

22.07 Git (практика)

00:56:45 — 02:05:01

Домашнее задание.

Выполнить команды:

- git init
- git status
- git add .
- git commit
- git log
- git branch



Мини-тест

1. Что делает git add?
2. Что делает git commit?
3. Что показывает git status?
4. Как создать новую ветку?
5. Как переключиться между ветками?

23.07 PostgreSQL (теория)

02:05:01 — 03:25:10

Домашнее задание.

Установить PostgreSQL. Создать первую базу данных.



Дополнительно Изучить:

- SELECT
- INSERT
- UPDATE
- DELETE



Собеседование

- Что такое SQL?
- Что такое Primary Key?
- Что такое Foreign Key?

ИТОГ НЕДЕЛИ

Перед переходом дальше обязательно знать:

- Git
- Commit
- Branch
- Merge
- Repository
- Основы SQL
- PostgreSQL

26.07	PostgreSQL (практика) Пагинация	03:25:10 — 05:07:51	 Практика • Создать таблицу users • Добавить 10–20 записей • Написать SELECT-запросы • Попробовать LIMIT и OFFSET  Дополнительно (15 минут) Как работает пагинация в REST API
			 Мини-тест 1. Что делает LIMIT? 2. Для чего OFFSET? 3. Почему пагинация важна? 4. Когда использовать ORDER BY?  Собеседование • Как реализовать пагинацию? • Почему нельзя отдавать миллион строк?
27.07	Миграции	05:07:51 — 06:19:38	 Практика • Понять назначение миграций • Создать первую миграцию • Применить её  Мини-тест 1. Что такое миграция? 2. Зачем они нужны? 3. Что такое rollback?  Собеседование • Использовал ли миграции? • Как обновляется БД на продакшене?
30.07	Операционная система Переменные окружения Makefile	07:01:14 — 07:46:29	 Практика • Создать файл .env • Подключить переменные окружения • Написать простой Makefile  Мини-тест

1. Что хранится в .env?
 2. Почему нельзя хранить пароль в коде?
 3. Для чего нужен Makefile?
-  Собеседование
- Что такое переменные окружения?
 - Для чего нужен Makefile?

31.07 Docker

07:46:29 — 09:49:22

 Практика

- Установить Docker
 - Запустить первый контейнер
 - Создать Dockerfile
-  Дополнительно Docker за 20 минут

 Мини-тест

1. Что такое контейнер?
2. Что такое образ?
3. Что делает Dockerfile?
4. Для чего docker compose?

 Собеседование

- Зачем Docker backend-разработчику?
- Чем контейнер отличается от виртуальной машины?

ИТОГ НЕДЕЛИ

Перед продолжением курса необходимо знать:

- PostgreSQL
- SQL
- LIMIT / OFFSET
- Пагинацию
- Миграции
- .env
- Makefile
- Docker

01.08 Системные сигналы
Логгирование

09:49:22 — 11:10:55

 Практика

- Добавить логгирование в проект
- Разобраться с graceful shutdown

 Мини-тест

1. Что такое SIGINT?
2. Что такое graceful shutdown?
3. Для чего нужны логи?



Собеседование

- Какие уровни логирования знаешь?
- Что происходит при Ctrl+C?

02.08 Архитектура Go
Backend
Заключение

11:10:55 — 11:54:06



Практика

- Повторить весь курс
- Составить план первого pet-проекта
- Подготовить GitHub



Мини-тест

1. Что такое Clean Architecture?
2. Что такое Handler?
3. Что такое Repository?
4. Что такое Service?
5. Как бы ты начал новый проект?



Собеседование

- Расскажи архитектуру своего проекта.
- Почему выбрал Go?
- Какие темы курса были самыми сложными?

КУРС ПОЛНОСТЬЮ ЗАВЕРШЁН

Теперь ты должен уверенно знать:

- Go Core
- Concurrency
- HTTP
- REST API
- JSON
- Git
- PostgreSQL
- SQL
- Docker
- Миграции
- Логгирование
- Архитектуру Go Backend

Следующий этап:

1. Сделать Todo API.
2. Сделать второй проект (например, интернет-магазин или сервис бронирования).
3. Оформить GitHub.
4. Составить резюме.
5. Начать проходить собеседования.